

Multi-Layer Neural Network for Three-Class Classification from Scratch

A study of Gaussian and spiral datasets using NumPy

Ankit Saini (2522MA01)
Tiyas Chakraborty (2522MA02)
Tanu Goliyan (2621MA05)
Murari Ray (2621MA07)

Department of Mathematics
Indian Institute of Technology Patna

16-08-2026

1 Introduction

This work implements a fully connected neural network (FCNN) directly with NumPy for a three-class classification task. The notebook studies two synthetic datasets with different geometric structures: three separated Gaussian clusters and three interleaving spiral arms. A linear classifier is also implemented as a baseline. The central purpose is to examine how network depth and width affect the ability to represent class boundaries.

2 Objectives

The objectives of this experiment are to:

- (i) generate and classify two synthetic three-class datasets;
- (ii) implement forward propagation, backpropagation, and stochastic gradient descent without a pre-built neural-network model;
- (iii) compare candidate hidden-layer architectures using three-fold cross-validation;
- (iv) evaluate the selected model on training, validation, and test sets; and
- (v) compare the FCNN with a linear classifier.

3 Datasets and Preprocessing

Each dataset contains 1,500 observations with two input coordinates and three class labels (0, 1, 2). The split is performed separately within each class, yielding 60% training, 20% validation, and 20% testing data.

Inputs are standardized using the training-set mean and standard deviation. The same training statistics are applied to validation and test inputs, avoiding the use of held-out data to estimate normalization parameters. Class labels are converted to three-dimensional one-hot vectors for training.

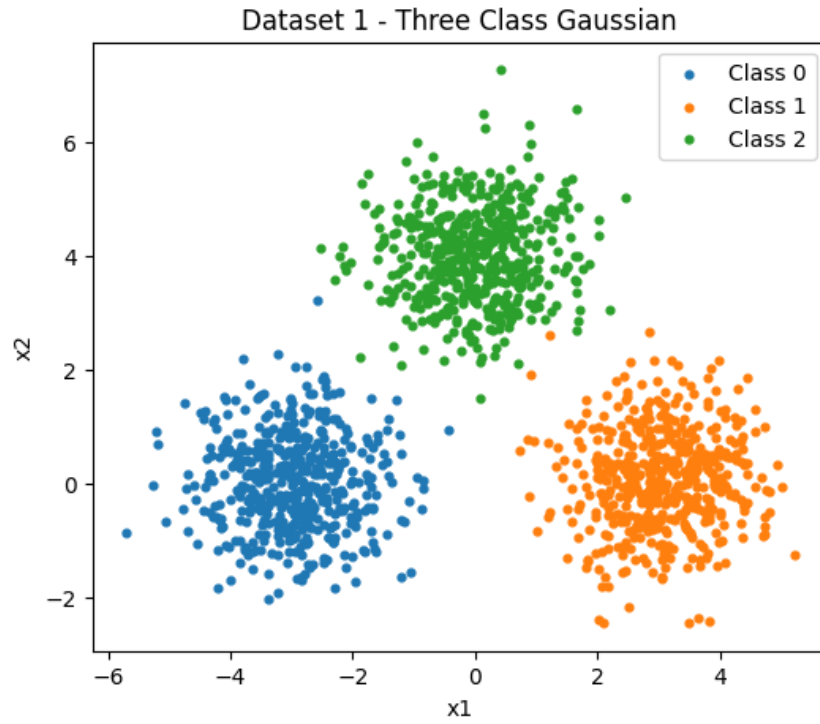


Figure 1: Visualization of Dataset 1 showing Gaussian clusters.

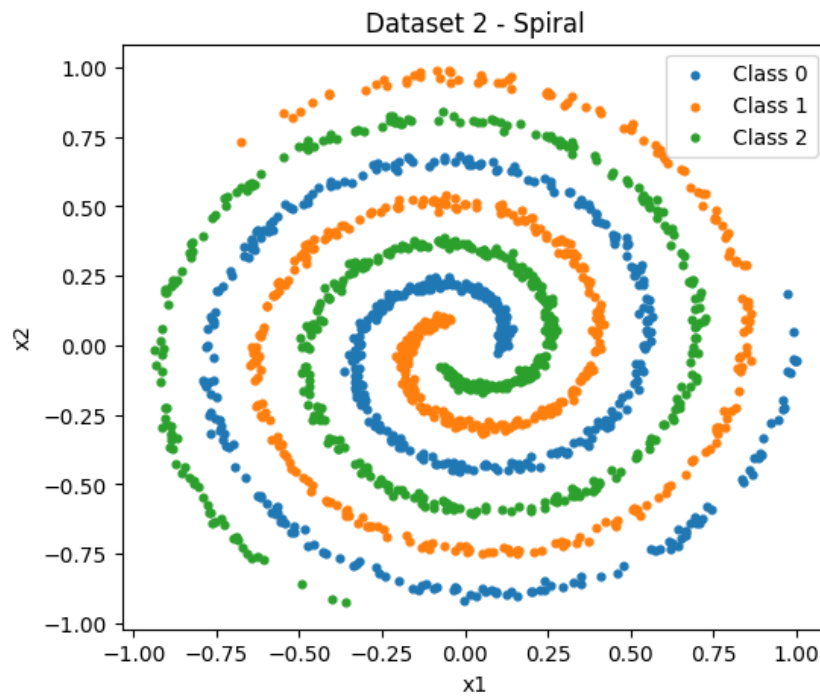


Figure 2: Visualization of Dataset 2 showing interleaving spiral arms.

Dataset	Construction
Dataset 1	Three Gaussian clusters centered at $(-3, 0)$, $(3, 0)$, and $(0, 4)$, with covariance $0.7 \cdot I$; 1,500 samples.
Dataset 2	Three noisy spiral arms; angle spans 0 to 4π and radius spans 0.1 to 1.0; 1,500 samples.

Subset	Count per dataset	Share
Training	900	60%
Validation	300	20%
Testing	300	20%

4 Methodology

4.1 Network Architecture

The FCNN receives two input features and produces three output scores. Hidden layers use the hyperbolic tangent activation, while the output layer is linear. The notebook tests the following architectures:

Dataset	Candidate hidden-layer widths	Selected architecture
Dataset 1	(4), (8), (16), (32)	(16)
Dataset 2	(8, 8), (16, 16), (32, 16), (32, 32)	(16, 16)

Dataset 1: $2 \rightarrow 16 \rightarrow 3$

- Input layer: 2 features
- Hidden layer: 16 neurons, tanh
- Output layer: 3 class scores

Dataset 2: $2 \rightarrow 16 \rightarrow 16 \rightarrow 3$

- Input layer: 2 features
- Hidden layer 1: 16 neurons, tanh
- Hidden layer 2: 16 neurons, tanh
- Output layer: 3 class scores

4.2 Forward Propagation

For an input vector $\mathbf{x}$, the hidden activations are computed successively as $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ and $a^{(l)} = \tanh(z^{(l)})$. The final layer produces the score vector $\mathbf{o} = W^{(L)}a^{(L-1)} + b^{(L)}$. The predicted class is the index of the largest output score, $\arg \max(\mathbf{o})$.

4.3 Loss and Backpropagation

For each training observation, the notebook uses half the sum of squared differences between the output score vector and the one-hot target:

$$E = \frac{1}{2} \sum_k (o_k - y_k)^2$$

Backpropagation starts with the output error $\delta^{(L)} = o - y$ and propagates it backward through each tanh hidden layer using the derivative $1 - \tanh^2(z)$. Parameters are updated after each sample using stochastic gradient descent.

4.4 Optimization and Model Selection

The random seed is 42. Training uses 200 epochs and a learning rate of 0.01. Candidate architectures are assessed by three-fold cross-validation on the training portion. The architecture with the highest mean fold accuracy is selected, then trained on the complete training portion and evaluated on the separate validation and test sets.

5 Experimental Configuration

Parameter	Setting
Number of classes	3
Input dimension	2
Samples per dataset	1,500
Train / validation / test	60% / 20% / 20% (900 / 300 / 300)
Random seed	42
Epochs	200
Learning rate	0.01
Hidden activation	tanh
Output layer	Linear scores
Training method	Sample-wise SGD with backpropagation
Architecture selection	3-fold cross-validation

6 Experimental Results

6.1 Dataset 1: Gaussian Clusters

The one-hidden-layer architecture with 16 neurons achieved the highest mean cross-validation accuracy (99.56%) among the four candidates. Its reported training accuracy was 99.56%, while validation and test accuracy were both 100%. The validation and test confusion matrices were perfectly diagonal, with 100 correctly classified observations in each class.

Architecture	Fold 1	Fold 2	Fold 3	Mean CV
(4)	99.33%	99.00%	99.67%	99.33%
(8)	99.67%	99.33%	99.00%	99.33%
(16)	99.67%	99.33%	99.67%	99.56%
(32)	99.00%	100.00%	99.33%	99.44%

Metric	Accuracy
Training	99.56%
Validation	100.00%
Test	100.00%

Test confusion matrix for the selected Dataset 1 model (rows are actual classes; columns are predicted classes):

	Predicted 0	Predicted 1	Predicted 2
Actual 0	100	0	0
Actual 1	0	100	0
Actual 2	0	0	100

Validation confusion matrix for the selected Dataset 1 model (rows are actual classes; columns are predicted classes):

	Predicted 0	Predicted 1	Predicted 2
Actual 0	100	0	0
Actual 1	0	100	0
Actual 2	0	0	100

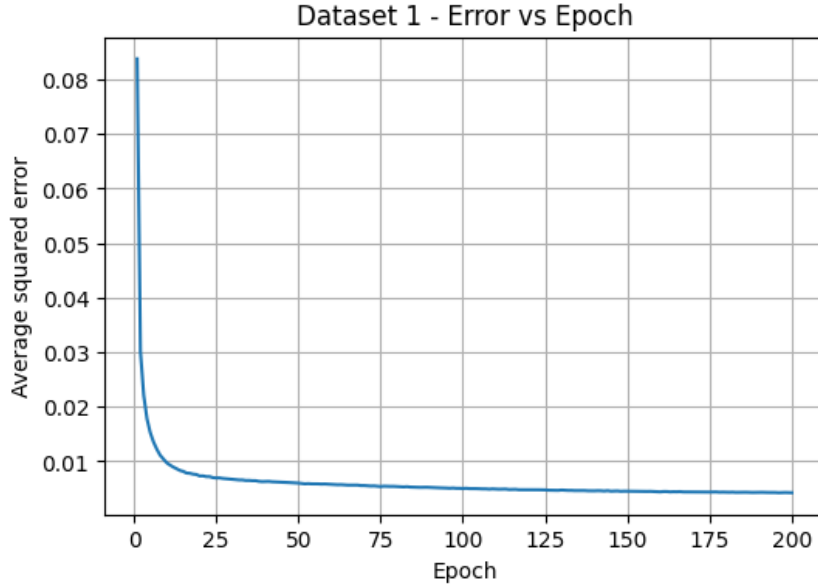


Figure 3: Average mean squared error Vs epoch for dataset 1

6.2 Dataset 2: Spiral Classes

The two-hidden-layer architecture (16, 16) achieved the highest mean cross-validation accuracy (53.67%). Its training, validation, and test accuracies were 63.56%, 59.33%, and 60.00%, respectively. The confusion matrices show that the spiral classes are substantially harder to separate than the Gaussian clusters.

6.3 Cross-Validation Comparison for Dataset 2

Architecture	Fold 1	Fold 2	Fold 3	Mean CV
(8, 8)	39.33%	50.00%	48.67%	46.00%
(16, 16)	53.33%	49.00%	58.67%	53.67%
(32, 16)	52.67%	52.00%	50.00%	51.56%
(32, 32)	56.00%	54.00%	48.00%	52.67%

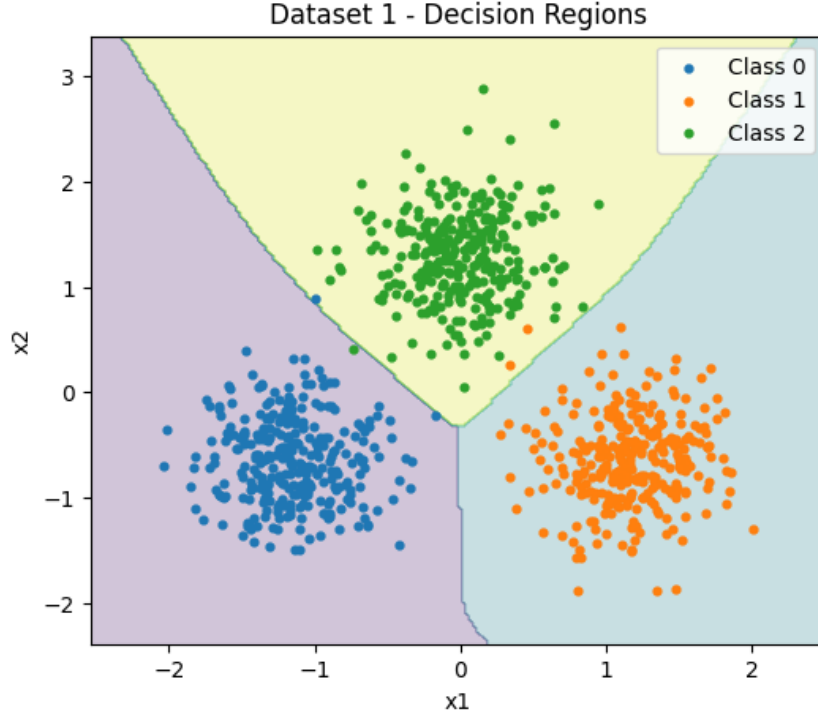


Figure 4: Decisions Boundries for dataset 1

Dataset 2 Metric	Accuracy
Training	63.56%
Validation	59.33%
Test	60.00%

Test confusion matrix for the selected Dataset 2 model (rows are actual classes; columns are predicted classes):

	Predicted 0	Predicted 1	Predicted 2
Actual 0	73	18	9
Actual 1	27	61	12
Actual 2	40	14	46

Validation confusion matrix for the selected Dataset 2 model (rows are actual classes; columns are predicted classes):

	Predicted 0	Predicted 1	Predicted 2
Actual 0	74	13	13
Actual 1	27	58	15
Actual 2	41	13	46

7 Comparison with a Linear Classifier

A linear model with two inputs and three output scores was trained using the same squared-error form and sample-wise updates. On Dataset 1, both the FCNN and linear model achieved

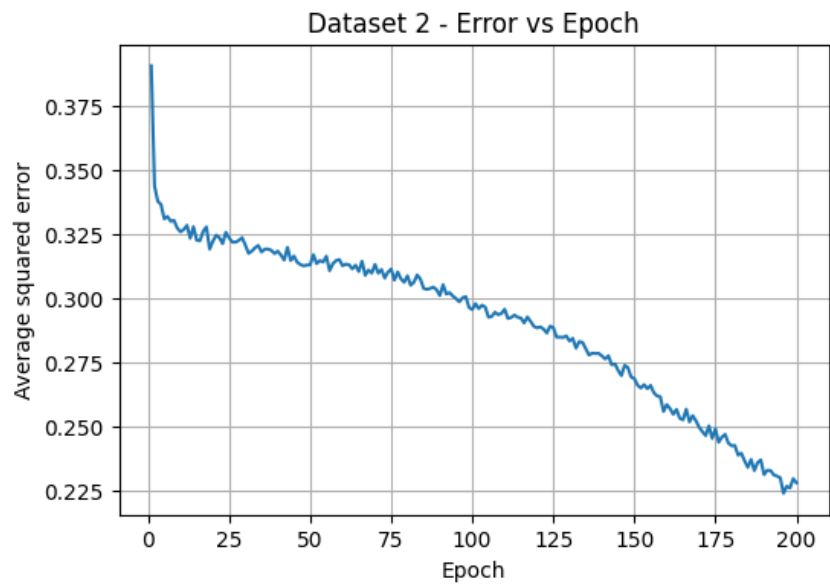


Figure 5: Average mean squared error Vs epoch for dataset 1

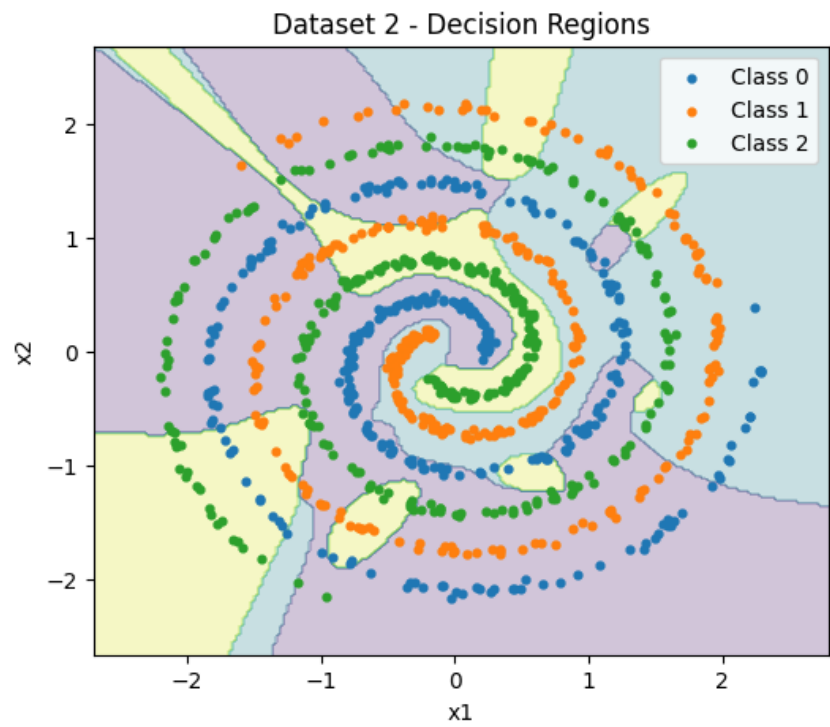


Figure 6: Decisions Boundries for dataset 1

100% validation and test accuracy. On Dataset 2, the FCNN substantially outperformed the linear baseline.

Dataset	Model	Validation accuracy	Test accuracy
Gaussian	FCNN	100.00%	100.00%
Gaussian	Linear	100.00%	100.00%
Spiral	FCNN	59.33%	60.00%
Spiral	Linear	36.33%	37.33%

The Gaussian classes are readily separated by linear decision boundaries, so a linear classifier is sufficient for this dataset. The spiral geometry is nonlinear; the multilayer network captures more of this structure, although its accuracy remains moderate under the notebook’s fixed training setup.

8 Discussion

The experiments illustrate that model complexity should be matched to the geometry of the data. For the Gaussian dataset, increasing the hidden width beyond 16 neurons did not improve the cross-validation score meaningfully, and a linear baseline was already sufficient. For the spiral dataset, adding a second hidden layer improved performance over the linear classifier, but the selected network achieved only 60% test accuracy.

The training accuracy for the spiral model (63.56%) is higher than its validation and test scores (59.33% and 60.00%), indicating a modest generalization gap. The results should be interpreted as a demonstration of the implemented learning procedure and architecture comparison, rather than as an exhaustive search for the best possible classifier.

The notebook also creates error-versus-epoch plots, decision-region plots, confusion matrices, and three-dimensional visualizations of hidden/output node responses. These diagnostics provide complementary views of convergence, class separation, and learned representations.

9 Conclusion

A fully connected neural network was implemented from scratch using NumPy for three-class classification on Gaussian and spiral datasets. The implementation includes forward propagation, tanh activations, squared-error loss, backpropagation, and stochastic gradient descent. Three-fold cross-validation selected a 16-neuron hidden layer for the Gaussian data and two 16-neuron hidden layers for the spiral data. The FCNN achieved 100% test accuracy on the Gaussian dataset and 60% on the spiral dataset, outperforming the linear baseline on the latter. Overall, the experiment demonstrates the importance of nonlinear representations for data with curved class boundaries.

10 Key Experimental Values

Item	Value
Datasets	Three-class Gaussian; three-arm spiral
Samples per dataset	1,500
Input dimension	2
Train / validation / test	900 / 300 / 300
Activation	tanh hidden layers; linear output
Loss	Half sum of squared errors
Epochs / learning rate	200 / 0.01
Best Gaussian architecture	(16,), mean CV 99.56%
Gaussian test accuracy	100.00%
Best spiral architecture	(16, 16), mean CV 53.67%
Spiral test accuracy	60.00%
Linear baseline test accuracy	Gaussian 100.00%; spiral 37.33%